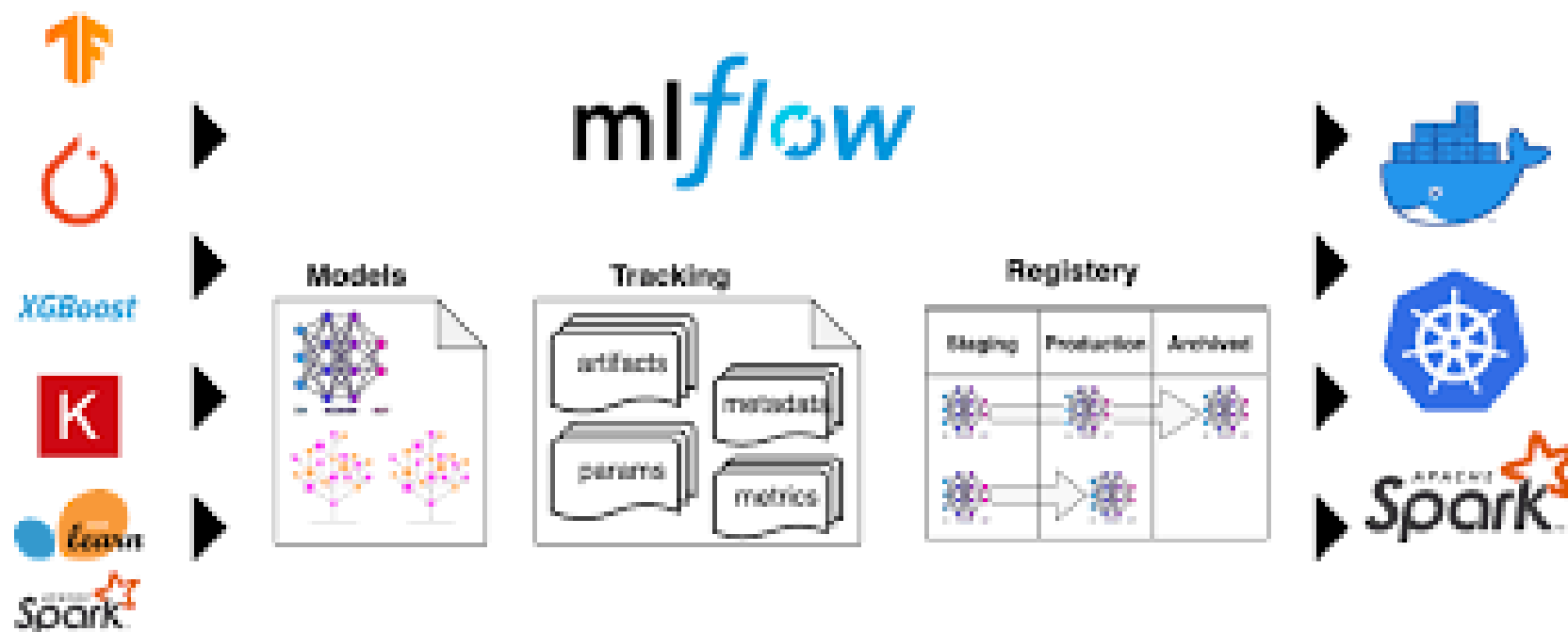
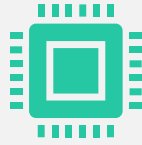




Gestión de Experimentos en Machine Learning

¿Qué es MLflow?



Es una **plataforma de código abierto** para gestionar el ciclo de vida de modelos de Machine Learning.



Facilita la **experimentación**, **reproducibilidad** y **despliegue** de modelos.



Compatible con **Scikit-Learn**, **TensorFlow**, **PyTorch** y más.

Componentes Principales de Mlflow

MLflow tiene 4 módulos clave:

MLflow Tracking

Registra experimentos, métricas, parámetros y modelos.

Permite comparar resultados fácilmente en una interfaz gráfica.

◆ Ejemplo: `mlflow.log_metric("accuracy", 0.92)`

Compatible con Scikit-Learn, TensorFlow, PyTorch y más.

Componentes Principales de MLflow

MLflow Projects

Define entornos reproducibles con
código + dependencias.

Usa **archivos conda.yaml** o **Docker**
para gestionar versiones.

◆ *Ejemplo:* `mlflow run my_project -`
`P alpha=0.5`

Componentes Principales de MLflow

MLflow Models

Estandariza el guardado de modelos en **formato universal (MLmodel)**.

Facilita la **carga y reutilización** en diferentes frameworks.

◆ *Ejemplo:* `mlflow.sklearn.log_model(model, "my_model")`

Componentes Principales de MLflow

MLflow Registry

Permite el **versionado y despliegue de modelos** en producción.

Integra modelos con **API REST** y **CI/CD**.

◆ *Ejemplo:* `mlflow models serve -m my_model`

Instalación

pip install mlflow

```
C:\WINDOWS\system32>mlflow ui
```

```
ng on http://127.0.0.
```

```
eue:Task queue depth
```

```
eue:Task queue depth
```

```
eue:Task queue depth
```

```
eue:Task queue depth
```

```
eue:Task queue depth
```

```
eue:Task queue depth
```

```
eue:Task queue depth
```

```
C:\WINDOWS\system32>mlflow ui
```

Ejemplo

```
> mlflow ui
```

➤ Abre en el navegador

http://127.0.0.1:5000

ng on <http://127.0.0.1>

eue:Task queue depth

ueue:Task queue depth

ueue:Task queue depth

```
eue:Task queue depth
```

```
eue:Task queue depth
```

ueue:Task queue depth

```
eue:Task queue depth
```

```
import mlflow

mlflow.set_tracking_uri("http://127.0.0.1:5000")
mlflow.set_experiment("mi_experimento")

with mlflow.start_run(run_name="run_con_servidor"):
    mlflow.log_param("depth", 6)
    mlflow.log_metric("auc", 0.81)
```

Experiments



Default



mi_experimento



mi_experimento

[Provide Feedback](#)[Add Description](#)[Share](#)

Runs

Evaluation

Experimental

Traces

[+ New run](#)

Time created ▾

State: Active ▾

Datasets ▾

Sort: Created ▾

Columns ▾

Group by ▾



Run Name

Created ▾

Dataset

D



● run_con_servidor



15 seconds ago

-

11

Runs

intrigued-fish-949



Register model

Overview

Model metrics

System metrics

Traces

Artifacts

Search parameters

Parameter	Value
alpha	0.01

Search metrics

Metric	Value
MAE	1.1826851558379554
MSE	2.6147980548680065
R2	0.9545718935323327

¿Por qué usar MLflow?

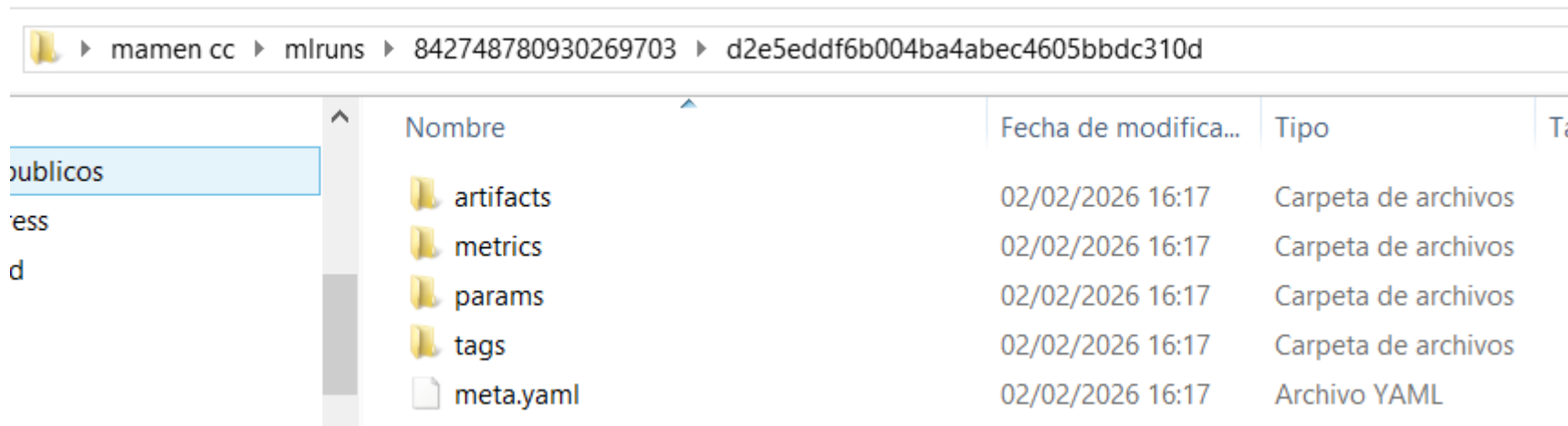
- ✓ Automatiza y centraliza el seguimiento de experimentos.
- ✓ Fácil integración con cualquier framework de ML.
- ✓ Historial completo de métricas y versiones de modelos.
- ✓ Facilita el despliegue con MLflow Models y Registry.

Starting a new experiment

```
import mlflow
# Create new Experiment
mlflow.create_experiment("My Experiment")
# Tag new experiment
mlflow.set_experiment_tag("scikit-learn", "lr")
# Set the experiment
mlflow.set_experiment("My Experiment")
```

Autolog()

```
mlflow.set_tracking_uri("file:./mlruns")  
mlflow.sklearn.autolog()
```



The screenshot shows a file explorer window with the path `mamen cc > mlruns > 842748780930269703 > d2e5eddf6b004ba4abec4605bbdc310d`. The main pane displays a table of files and folders. The left sidebar shows a tree view with 'publicos' selected. The table has columns for 'Nombre', 'Fecha de modifica...', 'Tipo', and 'T'. The files listed are 'artifacts', 'metrics', 'params', 'tags', and 'meta.yaml', all created on 02/02/2026 at 16:17. 'artifacts', 'metrics', 'params', and 'tags' are folders ('Carpeta de archivos'), while 'meta.yaml' is a file ('Archivo YAML').

Nombre	Fecha de modifica...	Tipo	T
artifacts	02/02/2026 16:17	Carpeta de archivos	
metrics	02/02/2026 16:17	Carpeta de archivos	
params	02/02/2026 16:17	Carpeta de archivos	
tags	02/02/2026 16:17	Carpeta de archivos	
meta.yaml	02/02/2026 16:17	Archivo YAML	

```
2026/02/02 16:13:01 INFO mlflow.utils.autologging_utils: Created MLflow autologging run with ID '8f83d061d9bd4c89b90cb92b80e6166c', which will track hyperparameters, performance metrics, model artifacts, and lineage information for the current sklearn workflow  
2026/02/02 16:17:13 WARNING mlflow.models.model: Logging model metadata to the tracking server has failed. The model artifacts have been logged successfully under mlflow-artifacts:/842748780930269703/8f83d061d9bd4c89b90cb92b80e6166c/artifacts. Set logging level to DEBUG via `logging.getLogger("mlflow").setLevel(logging.DEBUG)` to see the full traceback.
```

Autolog parameters

Parameters (5)

Name	Value
copy_X	True
fit_intercept	True
n_jobs	None
normalize	deprecated
positive	False

```
# Model
```

```
lr = LinearRegression()  
lr.fit(X, y)
```

Autolog metrics

▼ Metrics (5)

Name	Value
training_mean_absolute_error 	6640.5
training_mean_squared_error 	89637953.1
training_r2_score 	0.952
training_root_mean_squared_error 	9467.7
training_score 	0.952

```
# Model
```

```
lr = LinearRegression()
```

```
lr.fit(X, y)
```

What is MLflow Tracking?

- Model Metrics
 - F1, Recall, Accuracy, MSE, etc...
- Parameters
 - library specific
- code
 - `train.py`
- other artifacts
 - tokenizers, pickle, etc...

Setting a training run variable

```
import mlflow
# Set experiment
mlflow.set_experiment("My Experiment")
# Start a run
run = mlflow.start_run()
# Print run info
run.info
```

Logging to MLflow Tracking

- **Metrics**

- `log_metric("accuracy", 0.90)`
- `log_metrics({"accuracy": 0.90, "loss": 0.50})`

- **Parameters**

- `log_param("n_jobs", 1)`
- `log_params({"n_jobs": 1, "fit_intercept": False})`

- **Artifacts**

- `log_artifact("file.py")`
- `log_artifacts("./directory/")`

Logging a run

```
import mlflow
# Set Experiment
mlflow.set_experiment("LR Experiment")

# Start a run
mlflow.start_run()

# Model Training Code here
lr = LogisticRegression(n_jobs=1)

# Model evaluation Code here
lr.fit(X, y)
score = lr.score(X, y)
```

```
# Log a metric
mlflow.log_metric("score", score)

# Log a parameter
mlflow.log_param("n_jobs", 1)

# Log an artifact
mlflow.log_artifact("train_code.py")
```

Search runs example

```
import mlflow
# Filter string
f1_score_filter = "metrics.f1_score > 0.60"
# Search runs
mlflow.search_runs(experiment_names=["Insurance Experiment"],
                   filter_string=f1_score_filter,
                   order_by=["metrics.precision_score DESC"])
```

Common Metrics

- Regression
 - mean squared error
 - root mean squared error
 - mean absolute error
 - r2 score
- Classification
 - precision score
 - recall score
 - f1 score
 - accuracy score

Common Parameters

```
MODEL.get_params()
```

```
# Train the model
lr = LinearRegression()
lr.fit(X, y)
# Get params
params = lr.get_params(deep=True)
params
```

```
{'copy_X': True, 'fit_intercept': True, 'n_jobs': None,
  'normalize': 'deprecated', 'positive': False}
```

Model API functions

```
# Save a model to the local filesystem  
mlflow.sklearn.save_model(model, path)
```

```
# Log a model as an artifact to MLflow Tracking.  
mlflow.sklearn.log_model(model, artifact_path)
```

```
# Load a model from local filesystem or from MLflow Tracking.  
mlflow.sklearn.load_model(model_uri)
```

Load model from MLflow Tracking

```
# Pass run_id as f-string literal
model = mlflow.sklearn.load_model(f"runs:{run_id}/tracking_path")
# Show model
model
```

Evaluation Example

Training Data

```
X_train, X_test, y_train, y_test = \
    train_test_split(X, y,
                    train_size=0.7, random_state=0)
```

Linear Regression model

```
lr = LinearRegression()
lr.fit(X_train, y_train)
```

Dataset

```
eval_data = X_test
eval_data["test_label"] = y_test
```

Evaluate model with Dataset

```
mlflow.evaluate(
    "runs:/run_id/model",
    eval_data,
    targets="test_label",
    model_type="regressor"
)
```

Predict

```
import pandas as pd

model_uri = f"runs:/run_regresion/model"      # 'model' = artifact_path donde guar
model = mlflow.sklearn.load_model(model_uri)  # carga estimador sklearn nativo :cor

# Ejemplo de datos (ajusta columnas/orden a tu entrenamiento)
X_new = pd.DataFrame([
    {"sepal length (cm)": 5.1, "sepal width (cm)": 3.5, "petal length (cm)": 1.4,
     "petal width (cm)": 0.2}
])

y_pred = model.predict(X_new)
print(y_pred)
```